DOCUMENTO INTERNO · CONFIDENZIALE

Turni 568
Roadmap Tecnica

Analisi bug, interventi sicurezza e sviluppi futuri

VERSIONE	PROGETTO SUPABASE	REPOSITORY	DEPLOY
1.0 – Aprile 2026	fnwuanftlhbqksldgafe	MatVitale6/568-platform	568-platform.vercel.app

Indice

1. **Riepilogo stato attuale** — cosa funziona, cosa è stato corretto
2. **Bug critici confermati** — cache notifiche, Safari Push
3. **Problemi di sicurezza aperti** — CORS, CSP, autorizzazione push
4. **Nuove funzionalità richieste** — notifiche email, Telegram, multi-swap
5. **Miglioramenti UX / performance** — polling rimosso, onboarding utenti
6. **Piano di sviluppo** — priorità, dipendenze, stime
7. **Dettaglio tecnico implementazioni** — architettura proposta

- Critico — blocca uso o dato
- Alto — impatta esperienza
- Medio — funzionalità mancante
- Basso / Già risolto

1 Riepilogo stato attuale

Cosa è stato completato e verificato in produzione

✅ L'applicazione è operativa su **568-platform.vercel.app** con Supabase come backend. Le funzionalità core — calendario turni, gestione dipendenti, richiesta cambio turno, push notifications Android/Chrome — sono state sviluppate, deployate e testate.

COMPONENTE	STATO	NOTE
Login / Auth JWT	✓ Attivo	Supabase GoTrue, JWT, RLS
Calendario turni	✓ Attivo	Realtime WebSocket, settimana navigabile
Gestione dipendenti (admin)	✓ Attivo	CRUD profiles + employees table
Richiesta cambio turno	✓ Attivo	RPC security-definer, RLS corretto
Push notifications Android/Chrome	✓ Attivo	VAPID, Edge Function send-push, service worker v3
PWA / installabile	✓ Attivo	manifest.webmanifest, logo, start_url=/calendar
SPA routing Vercel	✓ Attivo	vercel.json rewrites, no-cache headers
Push Safari / iOS	✗ Non funziona	Vedi sezione 2
Cache isolamento utenti	✗ Bug confermato	Notifiche di utente sbagliato dopo logout
Polling 15s	✓ Rimosso	Causa ri-rendering visibile, fix applicato
Notifiche email	○ Da fare	Vedi sezione 4
Notifiche Telegram	○ Da fare	Vedi sezione 4
Multi-swap (primo accetta)	○ Da fare	Vedi sezione 4

2 Bug Critici Confermati

Problemi verificati in produzione che impattano direttamente l'uso

---

B-01 — Notifiche arrivano all'utente sbagliato dopo logout/login

Critico · Bug · Sicurezza

Sintomo: Accedendo con Irene, attivando le push, poi disconnettendo e accedendo come Emma sullo stesso dispositivo, le notifiche push continuano ad arrivare come se fossero destinate a Irene.

Causa radice: Il service worker e la push subscription rimangono registrati nel browser con l'endpoint VAPID associato al profilo di Irene. Al logout, `disable_push_subscription()` viene chiamato ma agisce solo lato DB — il service worker non viene de-registrato. Al successivo login di Emma, viene chiamato `register_push_subscription()` che sovrascrive la subscription nel DB, ma il browser mantiene ancora la vecchia registrazione attiva del service worker di Irene (o la nuova subscription ha lo stesso endpoint fisico, causando ambiguità).

Impatto: Privacy — un utente riceve notifiche destinate a un altro. Su un dispositivo condiviso tra colleghi questo è inaccettabile.

Fix proposto:

- Al logout: chiamare `pushManager.getSubscription()` e poi `subscription.unsubscribe()` per de-registrare fisicamente il browser, non solo lato DB.
- Al login: registrare sempre una nuova subscription fresca, confrontare l'endpoint con quello nel DB e forzare sovrascrittura.
- Aggiungere lato Edge Function: validare che `profile_id` della subscription ricevuta appartiene all'utente che sta facendo la richiesta.

Sforzo: Basso (2-3h) · Priorità: Immediata · File: `usePushNotifications.js` , `AuthContext.jsx`

B-02 — Push Notifications non funzionano su Safari / iOS

Alto · Bug

Sintomo: Su Safari (iPhone/iPad/Mac), il pulsante "Attiva notifiche" non funziona o non richiede il permesso.

Causa radice: Apple ha aggiunto supporto Web Push solo in iOS 16.4+ (marzo 2023) e macOS Ventura. Ci sono tre vincoli aggiuntivi rispetto a Chrome/Firefox:

- **Solo come PWA installata** — su iOS le Web Push funzionano SOLO se l'app è stata aggiunta alla home screen (modalità standalone). Da browser Safari non è possibile.
- **Richiede gesture utente** — la richiesta del permesso deve essere eseguita direttamente dentro un evento click (non in un useEffect o timeout). Safari è più restrittivo di Chrome.
- **Service Worker scope** — Safari richiede che il service worker sia nella root del sito; il nostro `push-sw-v2.js` è in root ma la registrazione deve specificare correttamente lo scope.

Fix proposto:

- Rilevare se l'utente è su Safari con `navigator.vendor.includes('Apple')`.
- Se non è in modalità standalone (`window.navigator.standalone !== true`), mostrare un banner esplicativo: *"Su iPhone installa prima l'app (Condividi → Aggiungi a Home) per ricevere notifiche"*.
- Assicurarsi che la chiamata a `Notification.requestPermission()` avvenga sincronicamente dentro l'handler del click.

- Per macOS Safari, testare che il VAPID endpoint funzioni (Apple usa un endpoint diverso: `https://web.push.apple.com`).

Limitazione residua: Le push su Safari iOS non sono del tutto paragonabili a quelle Android in termini di affidabilità. Questo è un vincolo di piattaforma, non del nostro codice.

Sforzo: Medio (4-6h) · Priorità: Alta · File: `usePushNotifications.js`, UI notifica

B-03 — Onboarding utenti richiede 3 step manuali

Alto · UX

Sintomo: Per creare un nuovo utente l'admin deve: (1) creare il profilo dall'app, (2) invitarlo manualmente dalla dashboard Supabase, (3) eseguire una query SQL per collegare `auth_user_id` al profilo. Attualmente il bottone "Invita" nell'app non manda nessuna email reale.

Fix proposto:

- Creare un trigger PostgreSQL `on_auth_user_created` che al primo login cerca un profilo con la stessa email e setta automaticamente `auth_user_id`.
- Collegare il bottone "Invita" a `supabase.auth.admin.inviteUserByEmail(email)` tramite una Edge Function con service role key (la anon key non ha i permessi admin).

Sforzo: Medio (3-5h) · Priorità: Alta · File: SQL trigger, nuova Edge Function `invite-user`

3 Problemi di Sicurezza Aperti

Identificati durante l'analisi OWASP, non ancora risolti

⚠️ Questi problemi non bloccano l'uso dell'app ma rappresentano vulnerabilità reali. Andrebbero corretti prima di un rollout a utenti reali.

🔒 S-01 — CORS wildcard sull'Edge Function send-push

Alto · Sicurezza

Problema: L'header `Access-Control-Allow-Origin: *` permette a qualsiasi dominio web di invocare la funzione `send-push`. Anche se c'è validazione JWT interna, un attaccante con un JWT valido potrebbe inviare push a qualsiasi profilo.

Fix: Sostituire `*` con `https://568-platform.vercel.app` nell'header CORS della Edge Function.

Sforzo: Minuto (15 min) | File: `supabase/functions/send-push/index.ts`

🔒 S-02 — Nessuna validazione sull'autorizzazione push

Alto · Sicurezza

Problema: L'Edge Function `send-push` accetta come parametro `profileIds` e manda notifiche a chiunque. Un utente autenticato potrebbe tecnicamente inviare notifiche push a tutti i dipendenti, non solo a quelli coinvolti nel proprio cambio turno.

Fix: Nella Edge Function, dopo aver estratto l'utente dal JWT, verificare che i `profileIds` richiesti siano effettivamente coinvolti in uno swap_request dove il chiamante è requester o target.

Sforzo: Basso (1-2h) | File: `supabase/functions/send-push/index.ts`

🔒 S-03 — Content-Security-Policy assente

Medio · Sicurezza

Problema: Nessun header CSP nel `vercel.json`. In assenza di CSP, un potenziale XSS (anche derivante da dipendenze npm compromesse) potrebbe esfiltrare i JWT o i dati degli utenti.

Fix: Aggiungere in `vercel.json` un header `Content-Security-Policy` per limitare le origini da cui possono essere caricati script, stili e connessioni.

```
// vercel.json — aggiunta header CSP
{
  "key": "Content-Security-Policy",
  "value": "default-src 'self'; connect-src 'self' https://*.supabase.co wss://*.supabase.co;
script-src 'self' 'unsafe-inline'; style-src 'self' 'unsafe-inline'; img-src 'self' data: blob:;"
}
```

Sforzo: Minuto (20 min) File: `vercel.json`

S-04/07 — Header HTTP di sicurezza mancanti

Basso Sicurezza

Mancano in `vercel.json`: `X-Frame-Options: DENY` (anti-clickjacking), `X-Content-Type-Options: nosniff` (anti MIME sniffing), `Referrer-Policy: strict-origin-when-cross-origin`. Questi sono ignorabili nell'immediato ma consigliati.

Sforzo: 10 minuti File: `vercel.json`

4 Nuove Funzionalità Richieste

Feature proposte che richiedono sviluppo ex-novo

F-01 — Multi-swap: richiesta a più persone, primo che accetta vince

Alto | Feature

Richiesta: Un dipendente deve poter inviare la richiesta di cambio turno a più colleghi contemporaneamente. Quando uno accetta, gli altri vengono automaticamente annullati.

Impatto sul DB: Nessuna modifica allo schema — la tabella `swap_requests` già supporta righe multiple per lo stesso `shift_id`. Serve modificare le RPC e le policy RLS.

Cambiamenti richiesti:

- **RPC** `create_swap_request`: rimuovere il vincolo "una sola richiesta pendente per turno", accettare array di `target_employee_id`.
- **RPC** `respond_to_swap_request`: al momento dell'accettazione, fare un `UPDATE swap_requests SET status='cancelled' WHERE shift_id=... AND status='pending' AND id != accettata`.
- **UI** `Requests.jsx`: mostrare la lista dei destinatari di una richiesta multipla; evidenziare "in attesa di risposta da N persone".
- **Notifiche**: quando qualcuno accetta, notificare il richiedente + tutti gli altri destinatari che la richiesta è stata chiusa.

Edge case: Se due persone accettano simultaneamente (race condition), la RPC deve usare `SELECT ... FOR UPDATE` o una transazione per garantire che solo una accettazione vada a buon fine.

Sforzo: Alto (8-12h) | Priorità: Alta | File: SQL RPCs, `Requests.jsx`, `RequestsContext.jsx`

F-02 — Notifiche via Email

Medio | Feature

Motivazione: Le push web non funzionano su Safari, e su Android potrebbero non arrivare se le notifiche dell'app sono disabilitate nelle impostazioni del telefono. L'email è il canale più affidabile e universale.

Architettura proposta:

- Usare **Resend** (resend.com) — si integra nativamente con Supabase Edge Functions tramite API REST, piano gratuito 3.000 email/mese.
- Creare una nuova Edge Function `send-email` (o aggiungere logica in `send-push`) che accetta `{ to, subject, html }` e chiama l'API Resend.
- Nei casi di swap request (nuova richiesta, accettata, rifiutata) mandare email in parallelo alle push.
- Aggiungere un campo `email_notifications: boolean` nelle preferenze utente (tabella `profiles`) per permettere opt-out.

Template email minimo: oggetto strutturato con nome richiedente, data turno, link alla app per rispondere.

Sforzo: Medio (4-6h) | Dipendenza: account Resend + dominio verificato | File: nuova Edge Function, `RequestsContext.jsx`

F-03 — Notifiche via Telegram Bot

Medio Feature

Motivazione: Telegram è ampiamente usato. Un bot Telegram garantisce notifiche affidabili, visibili, con possibilità di risposta diretta. Funziona su qualsiasi dispositivo senza dipendere dal supporto PWA/browser.

Architettura proposta:

- Creare un bot Telegram via **@BotFather** → ottenere il `BOT_TOKEN`.
- Aggiungere un campo `telegram_chat_id: text` alla tabella `profiles`.
- Creare una pagina/flow nell'app per il collegamento: l'utente apre il bot su Telegram, il bot risponde con un codice temporaneo, l'utente lo incolla nell'app → l'app salva il `chat_id` nel profilo.
- Nella Edge Function `send-push` (o una nuova `send-notification`) aggiungere una chiamata `POST https://api.telegram.org/bot{TOKEN}/sendMessage` per ogni utente con `telegram_chat_id` impostato.

Pro Telegram vs Email: messaggio istantaneo, nessuno spam filter, possibile aggiungere pulsanti inline per accettare/rifiutare direttamente da Telegram (richiede webhook Telegram).

Sforzo: Alto (8-10h con inline buttons)

Dipendenza: bot Telegram creato, BOT_TOKEN in Supabase secrets

File: schema SQL, Edge Function, UI collegamento

5 Miglioramenti UX e Performance

Già aggiunto e ulteriori ottimizzazioni identificate

U-01 — Rimozione polling 15 secondi (RISOLTO)

Risolto · UX

Problema: `RequestsContext.jsx` e `useCalendar.js` eseguivano un re-fetch ogni 15 secondi, causando ri-rendering visibile dei componenti. Lo stesso osservato da un consulente esterno come "la pagina si ricostruisce periodicamente".

Fix applicato: Rimossi entrambi i `setInterval`. L'aggiornamento avviene ora solo tramite Realtime WebSocket (Supabase) e all'evento `visibilitychange` (quando si torna sulla tab/app). Zero polling attivo.

U-02 — Indicatore di stato notifiche più chiaro

Medio · UX

Il pulsante "Attiva notifiche" non dà feedback sullo stato attuale su Safari/iOS. Proposto: mostrare stati distinti — *Non supportato*, *Installa prima l'app*, *Attive*, *Disabilitate dal sistema* — con istruzioni contestuali.

Sforzo: Basso (1-2h) | File: `usePushNotifications.js`, componente UI

U-03 — Onboarding utenti automatizzato

Basso · UX

Vedi B-03. Il trigger PostgreSQL `handle_new_user` risolverebbe in automatico il collegamento `auth_user_id` al profilo senza richiedere SQL manuale. Collegare il bottone "Invita" a una Edge Function con service role key per mandare email di invito reale.

Sforzo: Medio (3-4h) | File: SQL trigger, Edge Function `invite-user`

6 Piano di Sviluppo

Priorità, dipendenze e stima sforzo per ogni intervento

Sprint 1 — Correzioni immediate (1-2 giorni)

Da eseguire prima di aprire l'app a utenti reali.

ID	TITOLO	PRIORITÀ	SFORZO	FILE CHIAVE
B-01	Fix cache notifiche utente sbagliato	Critico	2-3h	`usePushNotifications.js`
S-01	CORS wildcard → dominio specifico	Alto	15 min	`send-push/index.ts`
S-02	Validare profileIds rispetto a swap_requests	Alto	1-2h	`send-push/index.ts`
S-03	Aggiungere header CSP in vercel.json	Medio	20 min	`vercel.json`

Sprint 2 — Bug importanti (3-5 giorni)

ID	TITOLO	PRIORITÀ	SFORZO	NOTE
B-02	Support Safari push (banner + detection)	Alto	4-6h	Vincolo iOS: solo da PWA installata
B-03	Onboarding utenti automatizzato	Alto	3-5h	Trigger SQL + Edge Function invite
F-01	Multi-swap primo accetta vince	Alto	8-12h	Race condition da gestire

Sprint 3 — Nuovi canali di notifica (1-2 settimane)

ID	TITOLO	PRIORITÀ	SFORZO	DIPENDENZA ESTERNA
F-02	Notifiche Email via Resend	Medio	4-6h	Account Resend + dominio
F-03	Notifiche Telegram Bot	Medio	8-10h	Bot Telegram, BOT_TOKEN secret
U-02	UI notifiche migliorata (stati Safari)	Medio	1-2h	—
S-04	Header sicurezza aggiuntivi	Basso	10 min	—

Critici risolti 1 / 2

Bug risolti 1 / 4

Feature implementate 0 / 3

7 Dettaglio Tecnico Implementazioni

Architettura proposta per le feature principali

F-01 — Architettura Multi-swap

Il flusso attuale permette una sola richiesta pendente per turno. Va esteso senza rompere il comportamento esistente.

Utente A
Seleziona turno + N colleghi

→

RPC create_swap_request
Inserisce N righe in swap_requests (una per target)

→

send-push
Notifica tutti gli N colleghi

Collega B accetta
Prima risposta

→

RPC respond_to_swap_request
Accetta la riga di B, annulla tutte le altre (stesso shift_id)

→

Notifiche
A: "B ha accettato". C,D,E: "Non più necessario"

```
-- RPC aggiornata: respond_to_swap_request con cancellazione parallela
BEGIN;
  -- 1. Accetta la richiesta scelta
  UPDATE swap_requests
  SET status = 'accepted', responded_at = now()
  WHERE id = p_request_id
  AND status = 'pending'
  AND target_employee_id = auth.uid()
  FOR UPDATE; -- evita race condition

  -- 2. Annulla tutte le altre richieste sullo stesso turno
  UPDATE swap_requests
  SET status = 'cancelled'
  WHERE shift_id = (SELECT shift_id FROM swap_requests WHERE id = p_request_id)
  AND id != p_request_id
  AND status = 'pending';
COMMIT;
```

F-02 + F-03 — Architettura Notifiche Multi-canale

Proposto: una Edge Function unificata `send-notification` che gestisce tutti i canali in parallelo.

RequestsContext
invoke('send-notification', { profileIds, title, body, url })

→

Edge Function send-notification
Recupera profili (push_endpoint, email, telegram_chat_id)

Web Push (VAPID)
Android / Chrome / Desktop

Resend API
Email HTML → tutti i dispositivi

Telegram Bot API
Messaggio istantaneo + pulsanti inline

```
// send-notification/index.ts — struttura logica
const results = await Promise.allSettled([
  pushEnabled ? sendWebPush(subscription, payload)     : null,
  emailEnabled ? sendEmail(email, subject, html)          : null,
```

```
  telegramEnabled ? sendTelegram(chat_id, message)         : null,
])
```

B-01 — Fix logout push subscription

```
// usePushNotifications.js — da aggiungere al logout
const unsubscribeOnLogout = async () => {
  const registration = await navigator.serviceWorker.ready
  const subscription = await registration.pushManager.getSubscription()
  if (subscription) {
    await subscription.unsubscribe()   // rimuove fisicamente dal browser
  }
  await supabase.rpc('disable_push_subscription')  // disattiva nel DB
}
```

ℹ️ **Nota sul Telegram Bot:** Per implementare la risposta diretta da Telegram (accettare/rifiutare il cambio dal messaggio), è necessario configurare un Webhook Telegram che punta a una Edge Function. Questo aumenta lo sforzo a ~12-15h ma rende il sistema molto più comodo per l'utente mobile.

⚠️ **Ordine di esecuzione consigliato:** Prima B-01 (privacy), poi S-01 (CORS) e S-03 (CSP) — sono operazioni di 15-20 minuti ciascuna. Poi F-01 (multi-swap) che è la feature più richiesta. Email e Telegram come step successivo una volta che il flusso di swap è stabile.